

Trippy AI

Developer's Implementation Guide

Technical Documentation for Intelligent Itinerary Orchestration

1. System Overview

Trippy AI is a full-stack travel planning platform that utilizes Large Language Models (LLMs) to generate structured, component-ready itineraries. Unlike traditional "one-shot" AI tools, Trippy utilizes a conversational agent that allows users to iteratively refine their plans through a chat interface.

2. Core Tech Stack

- **AI Engine:** Google Gemini Flash Lite (selected for low-latency reasoning).
- **Frontend:** React.js initialized with Vite for rapid development.
- **Backend:** Express.js (Node.js) acting as a secure API proxy.
- **Geospatial:** Google Places API (integrated via Autocomplete SDK).

3. Project Architecture

The repository follows a modular structure to separate AI logic from the user interface:

- `/web` : The React client. Contains the UI components and the `TripsProvider` context.
- `/backend` : The server. Handles routing and environment variable management.
- `/gemini-agent` : The "Brain" of the system. Contains the prompt engineering logic.

4. AI Implementation Deep-Dive

Structured Data Generation (`trippyCore.js`)

To prevent the model from returning unstructured markdown text (which would break the UI), the system forces Gemini Flash Lite to behave like a structured Data API by defining a specific object structure for responses.

```
// Example Implementation logic from gemini-agent/trippyCore.js
// Defining the expected JSON structure for itineraries
const itineraryStructure = {
  budgetBreakdown: {
    accommodation: "String",
    food: "String",
    activities: "String"
  },
  itinerary: [
    {
      day: "Number",
      morning: "String",
      afternoon: "String",
      evening: "String"
    }
  ]
};
```

The Refinement Agent (`agent.js`)

The implementation of the ChatPanel relies on the `chatWithAgent` function. It maintains an array of message history, allowing the user to provide feedback (e.g., "Make day 2 more affordable"). The agent processes this feedback and returns a new JSON object to update the frontend state.

5. Frontend & State Management

Global State (`TripsProvider.jsx`)

State is managed via the React Context API. This allows the `ItineraryPanel` (which displays the trip) and the `ChatPanel` (which modifies the trip) to stay perfectly in sync without prop-drilling.

Component Highlights

- **`ItineraryPanel.jsx`:** Dynamically maps through the "day" objects returned by the AI. It handles the rendering of budget categories and activity descriptions.
- **`ChatPanel.jsx`:** Manages a local UI stream for user messages and displays the "thinking" status of the Gemini Flash Lite model.
- **`googlePlacesAutocomplete.js`:** Provides real-world grounding by ensuring users select valid destinations before the AI generation begins.

6. Setup & Deployment

Local Development Environment

Environment Variables: Create a `.env` file in the root directory:

```
GEMINI_API_KEY=[Your Key]
```

Installation:

- Run `npm install` in the root folder.
- Run `npm install` inside the `/web` folder.

Execution:

Run `npm run dev` from the root to start the Express backend and Vite frontend concurrently.

Production Considerations

- **Rate Limiting:** The current backend implementation does not include rate limiting; this should be added before public deployment to manage Gemini API costs.
- **Error Boundaries:** A frontend Error Boundary should be implemented to handle network timeouts or malformed AI responses gracefully.

7. Future Roadmap

- **Persistence:** Integration of a database (Supabase/PostgreSQL) to allow users to save and retrieve past itineraries.
- **Map Visualization:** Integrating the Google Maps JavaScript API to plot the AI-generated activities on an interactive map.
- **Real-Time Pricing:** Connecting to flight and hotel APIs to provide live pricing data instead of AI estimates.

Generated by the Trippy AI Engineering Team